

CO₃: AT₂

**SIMATS ENGINEERING
ASSESSMENT TOOLS**

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO3: Construct regular expressions, and use the pumping lemma and closure properties to analyze regular languages. (BL:3)

Assessment Tool Weightage: 40%

QUESTIONS: 5
Duration : 1 Hour

Total Marks:25

Experiments

Code of Conduct:

I V. Smylika (192372115) (Name / Reg No) certify that this submission is my original work and adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Smylika ✓

Signature of the student:

$S \rightarrow 0A1, A \rightarrow 0A/1A1$
 This language generates strings that start with 0 and end with 1, with any combination of 0 and 1 in between.
 Ex: 01, 001, 011, 0101

code:

```

#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int n, r, valid = 1;
    printf("Enter a binary string:");
    scanf("%s", str);
    n = strlen(str);
    if (n < 2 || str[0] != '0' || str[n-1] != '1') {
        valid = 0;
    }
    for (i = 0; i < n; i++) {
        if (str[i] != '0' && str[i] != '1') {
            valid = 0;
            break;
        }
    }
    if (valid)
        printf("String belongs to the CFG.\n");
    else
        printf("String does not belong to the CFG.\n");
    return 0;
}
  
```


Q. CFG: $S \rightarrow 0S0 \mid 1S1 \mid 011 \mid 101$

This grammar generates palindromes over $\{0, 1\}$.

Ex: 0, 1, 00, 11, 010, 101, 0110

```
code: #include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int i, j, valid = 1;
    printf("Enter a binary string: ");
    scanf("%s", str);
    i = 0;
    j = strlen(str) - 1;
    while (i < j) {
        if (str[i] != str[j]) {
            valid = 0;
            break;
        }
        if ((str[i] != '0' && str[i] != '1') || str[j] != '0' && str[j] != '1') {
            valid = 0;
            break;
        }
        i++;
        j--;
    }
    if (valid)
        printf("String belongs to the CFG.\n");
    else
        printf("String does not belong to the CFG.\n");
    return 0;
}
```


$S \rightarrow 0S0 \mid A, A \rightarrow 1A1 \mid \epsilon$

this grammar generates strings of the form:
 $0^n 1^m 0^n$ where $n, m \geq 0$

Ex: 1, 111, 010, 01110, 0011100

program:

```
#include <stdio.h>
#include <string.h>
int main() {
    char str[100];
    int n, i, left = 0, right, valid = 1;
    printf("Enter a binary string:");
    scanf("%s", str);
    n = strlen(str);
    right = n - 1;
    for (i = 0; i < n; i++)
    {
        if (str[i] != '0' & str[right] != '1')
        {
            valid = 0;
        }
        right--;
    }
    if (valid)
        printf("string belongs to CFG.\n");
    else
        printf("string does not belong to the CFG.\n");
    return 0;
}
```

5. $S \rightarrow 0S0 \mid A, A \rightarrow 0A1 \mid 1A1 \mid \epsilon$


```
#include <stdio.h>
#include <string.h>
int main()
```

```
{
    char str[100];
    int n, i, valid = 0;
```

```
    printf("Enter a binary string:");
```

```
    scanf("%s", str);
```

```
    n = strlen(str);
```

```
    for (i = 0; i < n; i++) {
```

```
        if (str[i] != '0' && str[i] != '1') {
```

```
            printf("String does not belong to the CFG.\n");
            return 0;
```

```
        }
```

```
    }
    for (i = 0; i < n; i++)
```

```
    {
        if (str[i] == '1' &&
```

```
            str[i+1] == '0' &&
```

```
            str[i+2] == '1') {
```

```
                valid = 1;
```

```
                break;
```

```
            }
```

```
    }
    if (valid)
```

```
        printf("String belongs to the CFG.\n");
```

```
    else
```

```
        printf("String does not belong to the
```

```
        return 0;
```

```
}
```


SIMATS ENGINEERING ASSESSMENT TOOLS

CRITERIA FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92/100

[Handwritten signature]